Debugging Exercise: NumPy Shapes, Loops, and Breakpoints

Setup

You are given two files:

- `student_scores.py` — a module with two functions:
  - `compute_final_grades(scores, weights)` — uses NumPy matrix math to compute each student's final grade as a weighted average.
  - `find_first_failing_student(final_grades, student_names)` — uses a plain Python `for` loop with a `break` to find the first student (in roster order) whose final grade is below `PASSING_GRADE`.
- `report.py` — a runnable script (don't edit this!) that builds a class roster, calls both functions, and prints a report.

Run the script:

```
python report.py
```

It currently **crashes**. There are **2 bugs** in `student_scores.py`:

- **Bug 1** — a NumPy array **shape** problem.
- **Bug 2** — a `for`-loop `break` condition that's wrong.

Your job: find and fix both bugs so `python report.py` runs cleanly and produces the correct report (see Expected Output below).

Do not change `report.py` — only edit `student_scores.py`.

The Data

- `scores` is a matrix with **shape (8, 4)**: 8 students (rows) × 4 assignments (columns: HW1, HW2, Midterm, Final).
- `weights` is a vector with **shape (4,)**: one weight per assignment (0.15, 0.15, 0.30, 0.40), summing to 1.0.
- `PASSING_GRADE = 70.0`.

Bug 1: `compute_final_grades()` — a shape problem

Symptom: The script crashes with:

```
TypeError: unsupported format string passed to numpy.ndarray.__format__
```

right after printing:

```
final_grades.shape = (8, 1)  (expected: (8,))
```

How to debug

1. Open a Python shell (or add temporary `print()` statements) and inspect the shapes step by step:

   ```
   import numpy as np
   from student_scores import compute_final_grades
   import report

   print(report.scores.shape)     # (8, 4)
   print(report.weights.shape)    # (4,)
   ```

2. Look at the line in `compute_final_grades` that does:

   ```
   weights_col = weights.reshape(n_assignments, 1)
   final_grades = scores @ weights_col
   ```

   What shape does `weights_col` have? What shape does `scores @ weights_col` produce? Is that the shape we want?

3. **Questions to investigate:**

   - We want one number per student — shape `(8,)`. What shape should `weights` have (without reshaping) to make `scores @ weights` directly produce shape `(8,)`?
   - Why does reshaping `weights` into a column vector `(4, 1)` change the output shape to `(8, 1)` instead of `(8,)`?

4. **Fix:** remove the unnecessary reshape and use `weights` directly in the matrix multiplication.

Bug 2: `find_first_failing_student()` — a loop/break condition

Symptom: Once Bug 1 is fixed, the script runs without crashing — but it reports the **wrong** first failing student (it reports **Alice**, who actually has a passing grade of 89.45!).

How to debug — set a breakpoint

The function has a `for` loop over `(i, name)` pairs, with a comment:

```
# >>> SET A BREAKPOINT ON THE NEXT LINE <<<
```

1. Set a breakpoint on that line (in VS Code / PyCharm, click in the gutter; in plain Python, you can add a line `import pdb; pdb.set_trace()` right before it, or use `breakpoint()`).
2. Run `python report.py` (with Bug 1 already fixed) under the debugger, and **step through the loop one iteration at a time**. At each stop, inspect:
   - `i` — which student index are we on?
   - `name` — which student?
   - `grade` — what is `final_grades[i]`? Is it less than `PASSING_GRADE` (70.0)?
   - The current `if` condition: `if i >= 0:`
3. **Questions to investigate:**
   - Is `i >= 0` ever `False` for a normal `for` loop using `enumerate()`? What does that tell you about when this loop breaks?
   - We want to break out of the loop **only when we've found a student whose grade is below the passing threshold**. What comparison expresses that, using `grade` and `PASSING_GRADE`?
   - What should happen if **no** student is failing — should the loop ever break in that case? (Hint: think about what `first_failing_index` and `first_failing_name` should be left as.)
4. **Fix:** replace `if i >= 0:` with the correct condition involving `grade` and `PASSING_GRADE`.

Expected Output (after both fixes)

```
=== Final Grades ===
final_grades.shape = (8,)  (expected: (8,))
Alice: 89.45
Bob: 78.50
Carla: 93.75
Dinesh: 60.05
Elena: 87.60
```

```
Farid: 70.70
Grace: 95.95
Hugo: 59.85

=== First Failing Student (grade < 70.0) ===
First failing student: Dinesh (index 3), grade = 60.05
```

Note: Dinesh (index 3) is reported, **not** Hugo (index 7), even though Hugo also fails — because Dinesh comes first in roster order and the loop should `break` as soon as it finds him.

Bonus: Try a "no failures" case

After fixing both bugs, try changing one of Dinesh's and Hugo's scores in `report.py` so that every student passes (grade $\geq$ 70.0 for everyone), and re-run the script. You should see:

```
=== First Failing Student (grade < 70.0) ===
No failing students found.
```

If your fix to Bug 2 causes a crash or wrong output in this case, double-check that the loop only sets `first_failing_index` / `first_failing_name` **inside** the `if`, and that they correctly default to `None` when no student fails.

Reflection Questions

1. In Bug 1, `scores @ weights` and `scores @ weights.reshape(-1, 1)` both "work" without error in many cases (broadcasting is forgiving) — but produce different shapes. Why is it important to check `.shape` even when code doesn't crash?

2. In Bug 2, the buggy condition `if i >= 0:` is **always True** for `enumerate()` starting at 0. Why didn't Python raise an error for this "useless" condition? What does this tell you about the limits of relying on errors/crashes to find bugs?

3. When would you use a debugger breakpoint instead of `print()` statements to investigate a loop like this? What extra information does a breakpoint give you that a single `print()` might not?